Township: Technical Design Document

Repository	https://github.com/AmanSingh-tech/township
Live app	https://township-91vm.onrender.com/welcome. Demo sign-ins are listed on the sign-in page; codes appear on screen because SMS is mocked. Running it locally, every login and the demo reset are in deployment.md.
Client	Company A, Pune: a gated township of 2,400 flats in 28 towers
Author, date	Aman Kumar Singh, 5 November 2026

Figures marked **MEASURED** come from tests or the evaluation and can be reproduced with the commands in section 8; figures marked **TARGET** are the brief's goals or design aims. Assumptions are labelled A1–A12 (section 1.3). Appendix A checks the assignment point by point; Appendix B answers likely questions.

1. Executive summary and the client's problem

Company A runs a 2,400-flat township whose money and operations live in Excel, phone calls, 28 WhatsApp groups and paper gate registers. Township replaces those with one web app: residents pay maintenance, raise complaints and approve visitors from their phones; bank statements match themselves where the evidence is clear and wait for a person where it is not; the committee gets a monthly report built from an append-only ledger. The core build (resident login, bills with test payment, bank matching, complaints with assignment and status, the committee view) works end to end on the brief's sample data, is live, and is backed by 63 server tests, 6 browser tests and a reproducible evaluation.

Result	Measured	How
Bank credits posted to the wrong flat	0 of 140	Evaluation: 5 realistic statements with known flats
Payment matching time, against by hand	−88% (estimated)	Estimated on the 140-credit evaluation dataset using the time assumptions in A12; about −77% if confirming a suggestion takes 1 minute. The brief's target is −80% (section 5.3)
Complaint category right; right when the model says it is sure	81%; 100%	90 labelled complaints in English, Hindi, Marathi, Hinglish
Safety hazards escalated; false alarms	14 of 14; 0	Same complaint set
Requests at five times the bill-day peak, 95th percentile	under 28 ms	Load test at full size; hardware in section 8

1.1 The client's costs and targets, and what answers them

Brief's cost and target	What in the product addresses it	Where the committee sees it
₹64.8 lakh never collected; on-time payments 70% → 90%	Dues first on the home screen, with the reason; paying in two taps; alerts when bills issue; interest shown, never a surprise	"On-time payments" against 90%
₹8.64 lakh of staff time matching; matching time −80%	Credits with strong evidence post themselves; the rest arrive with evidence and the best three flats; re-uploads are harmless; UPI IDs are learned	"Payment matching time" against 20% of the time by hand

₹10.13 lakh of reopened complaints; reopened 25% → under 8%	The right technician from the start, a response timer, every step visible to the resident, duplicates linked into one job, closure only with the resident's rating	"Reopened complaints" against 8%

1.2 Scope

The core build is the priority and is held to the highest bar (sections 2, 3, 5, 6). **The rest of the brief's "must have"** (gate, amenity booking, notices, family members, the installable app) is built as optional depth, tested but secondary (section 9). **Out of scope**, as the brief says: accounting beyond maintenance (Tally stays, and the app exports to it), CCTV and barriers, a marketplace.

1.3 Assumptions

Given by the brief, so not assumptions: ₹6.50 per sq ft a month; the three sample flats; late fees under the bye-laws; visitor photos deleted after 90 days with a gate notice; English, Marathi and Hindi; mocked payments and messages; most bills paid in the first 10 days. Where the brief is silent, I chose; each choice is a setting or one function:

#	Assumption	Why
A1	"Days overdue" counts from the oldest unpaid due date, so T12-0408 (47 days) also owes the next month.	The only reading where the sample row and a monthly bill agree
A2	Late fee means simple interest on overdue principal; payments clear interest first, then the oldest bill. The 21% a year used is a configurable demonstration rate; the actual applicable rate must be confirmed against the federation's adopted bye-laws before production.	The brief says only "late fees per bye-laws"; simple interest on principal avoids interest on interest
A3	Bills are raised on the 1st and due on the 10th; opening arrears carry no interest in the app.	Matches the payment pattern; old arrears' interest cannot be recomputed reliably
A4	Bank statements are CSV exports; UPI narrations carry the payer name, UPI ID and UTR, sometimes a flat number.	How HDFC, SBI and ICICI exports look; columns are found by heading
A5	A credit posts automatically only on a flat number or registered UPI ID, a score of 80+, a lead of 30 over the next flat, and no rival strong evidence.	The brief: unclear payments are never posted automatically
A6	Response targets: 15 min critical, 2 h high, 8 h medium, 24 h low; one duty-manager number.	A starting point the admin can change
A7	Roles beyond the core two: accounts, committee, admin, guard.	The other must-haves, and "the committee approves", need them
A8	Money decisions need two different people: bill runs, manual posts, adjustments, login-number changes.	What makes a report trustworthy to a committee
A9	Receipts are numbered without gaps per financial year: `RCT/2026-27/00001`.	Indian practice; auditors look for gaps
A10	Each of the 4 gates has one tablet, paired once; guards share tablets and sign in with a PIN.	One point of entry per gate today
A11	One server is enough for one township: about 8 requests a second at the bill-day peak.	2,400 flats, most paying in 10 days
A12	Matching time: 2 minutes to match a credit by hand; with the app, 0 for one it posted, 0.5 to confirm a right top suggestion, 2 when the suggestion was wrong or the credit still waits. Also the evaluation's mix of payment styles.	Stated so the −88% can be checked and re-run with other numbers

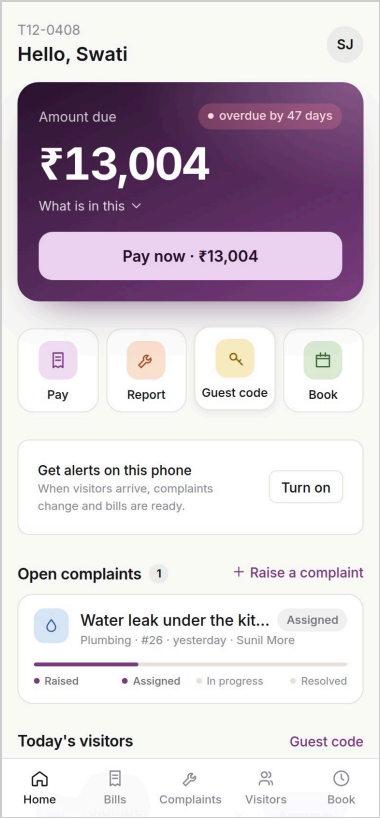
2. Core workflows delivered

Screen	Role	What it does
Sign in	All	Mobile number and a one-time code, linked to a flat (owner, tenant or family)
Resident home	Resident	Dues with a breakdown, open complaints and today's visitors; every action within two taps
Bills and test payment	Resident	Area × rate, late interest, a six-month chart, test-mode checkout, numbered receipt with PDF, ledger
Raise a complaint	Resident	Text and photo; AI category and urgency; safety rule
Complaint status	Resident	Timeline, assignee, response timer; rate the fix or reopen
Help desk queue	Help desk	Sorted by safety, urgency and deadline; auto-assignment; one-tap category change; linked duplicates
Bank matching	Accounts	The hard part: bank CSV in; confident credits post, the rest queue with evidence (section 5)
Committee report	Committee	Collections, dues ageing, complaint times and the brief's three measures; signed and frozen monthly

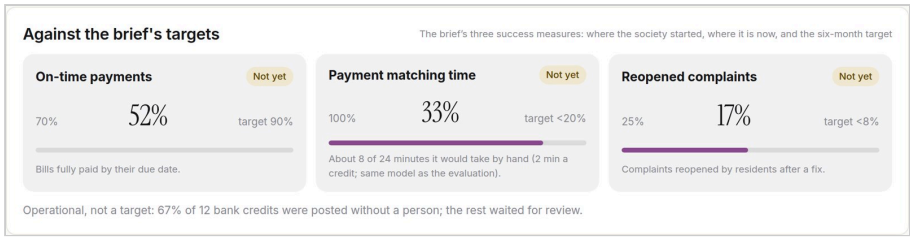
2.1 UI and design decisions

The brief asks for minimal design that counts as much as features: clean layouts, white space, a restrained palette, clear type, nothing without a purpose, every action within two taps. The choices, and why:

- **One accent, for the next action.** Warm neutrals with a lilac button and plum links; six soft tints only to tell categories, people and amenities apart; green, amber and red reserved for state. The eye finds the one thing to do.
- **The home screen is the brief's sentence:** dues (with "What is in this" for the breakdown), four quick actions, open complaints with named steps, today's visitors with approve in one tap.
- **Interactions that explain:** a tapped month in Bills shows its sum (980 sq ft × ₹6.50); a waiting visitor pulses; guest codes count down. CSS only, and off when the device asks for reduced motion.
- **Type for moments:** a display serif for greetings, titles and big amounts; Inter for reading. **Themes:** light, and a warm dark mode. **States:** skeletons while loading, written empty states, errors that say what to do. **Phone first**, with a sidebar and two columns from 960 px.
- **The desk is denser,** for a working day; the committee report leads with the brief's three measures, from their baselines.



Resident home: dues first, with overdue days and interest stated.



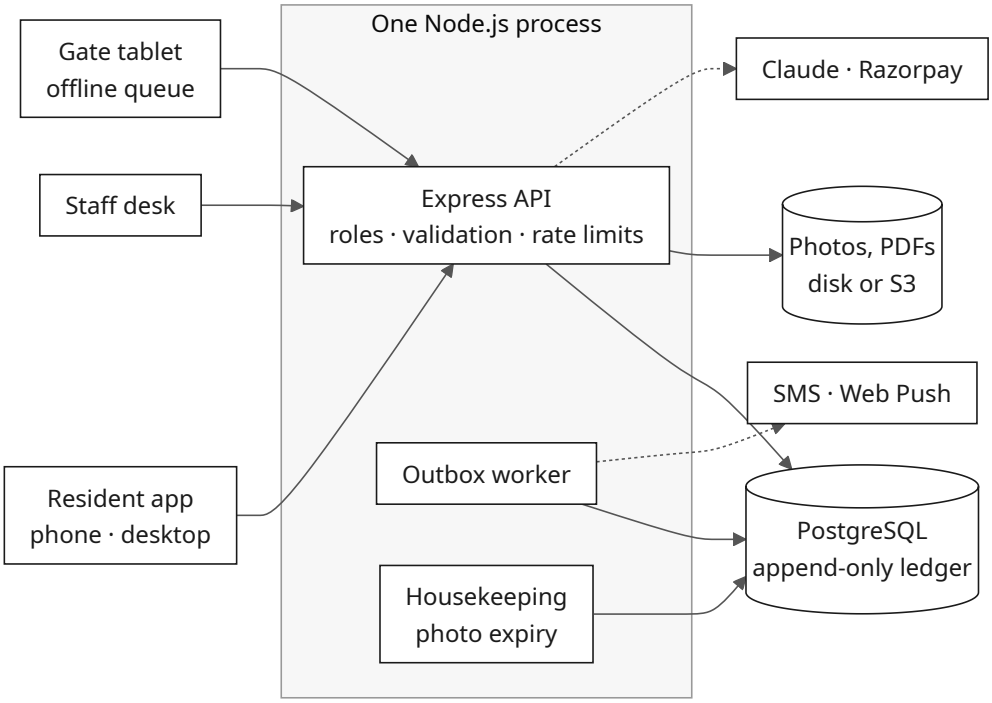
Committee report: the brief's three success measures, each from its baseline to its target, with automatic matching shown separately as an operational figure, not a target.

3. How we know each core workflow works

Core workflow	Implementation	Evidence (MEASURED, reproducible)
Resident sign-in	One-time codes, hashed, rate-limited, linked to a flat	`api.test`: single-use codes, lock after five wrong tries, three an hour, same reply for unknown numbers, expired sessions refused; every browser test signs in
Maintenance billing	Area × rate copied at issue; daily simple interest; interest first, then oldest bill; bill runs approved by a second person	`billing.test`: the brief's three flats match its table exactly; interest; allocation; exact rounding; bill runs. `governance.test`: issued only after approval, at the approved rate
Test payment	Server-priced order, signature check, the gateway's own record, one transaction for payment, ledger, receipt and alert	`api.test`: signature checked, amount from the gateway, recorded once (callback and webhook); `operations.test`: the test gateway; PDF receipt
Bank reconciliation	Evidence scoring with reasons; posts only on strong, unambiguous evidence; a second person for manual posts	`matching.test` and `reconciliation.test` (section 5.5); the evaluation: 0 wrong posts in 140 credits, an estimated −88% time (section 5.3)
Complaints with assignment and status	Classifier, safety rule, technician by category and tower, response timer, linked duplicates, rate or reopen	`complaints.test`; the browser test "complaint raised, resolved and rated"; the evaluation for triage (section 6)
Committee view	Report from the ledger; the brief's three measures; sign-off with a fingerprint	`governance.test`: a signed month is frozen and verified; a unit test pins the matching-time calculation; the browser test signs the report

Access control	Roles declared on every route; residents scoped to their flat; two-person rules in the database	`api.test`: roles reach only their areas; `reconciliation.test`: the database refuses self-approval

4. Architecture and technology decisions



React and Vite on the client; Node 20, Express and PostgreSQL 16 on the server; one Docker image, deployed on Render. Dotted lines are paid services, replaced by labelled placeholders when no credentials are set.

The shape follows from the numbers: one township, about 8 requests a second at peak, money that must not be wrong, and one developer. A single process with clear modules and one database gives transactions across everything that must change together (a payment, its ledger line, its receipt number and its alert), one thing to deploy, and nothing to keep in sync.

> **ADR-1 · One modular service, not microservices**
>
> **Context:** one township peaks at about 8 requests a second. **Decision:** one Node process with modules (billing, matcher, complaints, notify, amenities, report). **Consequences:** one deploy and no network between services; the app is stateless, so more processes can be added (section 4.4).

> **ADR-2 · PostgreSQL, with the ledger append-only in the database**
>
> **Context:** the committee needs a report it can trust. **Decision:** bills and payments write ledger lines in one transaction; triggers block updates and deletes; the app's database role cannot switch them off. **Consequences:** mistakes are corrected by new adjustment lines; `cli verify` checks the books balance and receipts have no gaps.

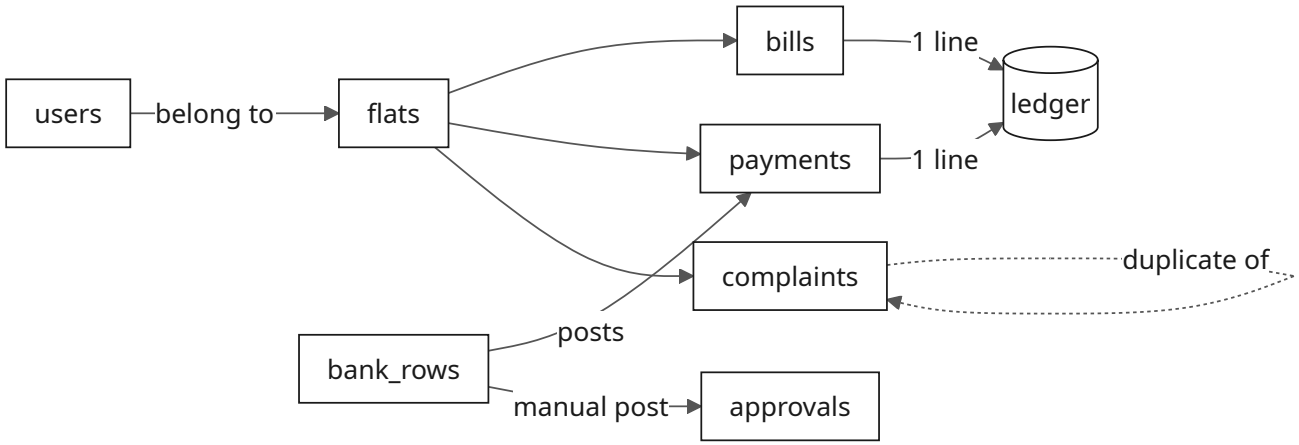
> **ADR-3 · Payment matching by explained rules, not a model**
>
> **Context:** a wrong match puts money on the wrong flat. **Decision:** score each credit with reasons; post only on strong, unambiguous evidence; queue the rest for a person, with a second person approving. **Consequences:** every post is repeatable and auditable.

> **ADR-4 · Complaint triage by LLM, with a rule above it and a local fallback**
>
> **Context:** a wrong category costs a tap; a missed gas leak is dangerous. **Decision:** safety words escalate by rule first; Claude classifies with a fixed schema; a local keyword model runs without a key or on any error. **Consequences:** a complaint is never blocked by the network, and the AI is never the last word on safety.

> **ADR-5 · Messages through a database outbox**
>
> **Context:** a crash must not lose a visitor alert, or send one for a payment that rolled back. **Decision:** messages are written in the same transaction as their cause and sent by a worker, urgent first. **Consequences:** no message queue to run; urgent alerts are not delayed by bulk ones.

> **ADR-6 · An installable web app with one-time-code sign-in**
>
> **Context:** residents open the app for seconds and forget passwords; the brief asks for an installable web app. **Decision:** one web app for phones, desktops and the gate tablet; codes to the registered number; guards use a PIN on a paired tablet. **Consequences:** no app store, one codebase.

4.1 Data model



The core of the schema; the full schema (34 tables) is in `server/migrations`, most of it for the optional depth in section 9. A flat's balance is always the sum of its ledger, never a stored number that could drift.

Table	Columns that matter, and why
bills	`period`; `area` and `rate` copied at issue, so a later rate change never rewrites an old bill; `amount`; `due_on`; unique per flat and month
payments	`amount`, `method`, `utr`, `gateway_payment_id` (unique), `receipt_no` (unique, gapless)
ledger	`kind` (bill, interest, payment, adjustment…), `debit`/`credit` in whole rupees, a link to what it records
bank_rows	`narration`, `utr`, `status`, `score`, `reasons` and `candidates` (the evidence shown to the treasurer)
complaints	`category`, `urgency`, the AI's own guess and confidence (kept to measure it), `escalated`, `duplicate_of`, `due_by`

Rules the database enforces, not only the app: triggers make `ledger`, `payments`, `audit_log` and `report_signoffs` append-only; `UNIQUE (flat_id, period)` stops double billing; `UNIQUE gateway_payment_id` records a payment once; `CHECK (decided_by <> requested_by)` stops self-approval; money is integer rupees with non-negative checks.

4.2 Low-level design

Modules. `server/routes/` is HTTP only (auth, resident, desk, gate, admin): each route declares its roles, validates input and calls `server/lib/`, which holds the rules: `billing`, `billruns`, `matcher`, `matchtime`, `approvals`, `complaints`, `classifier`, `report`, `notify`, `auth`, `audit`. `db.js` provides a pool and `tx()` (nested calls join the outer transaction). `web/src/` holds the resident, desk and gate pages, the front page, and shared UI.

A request's life. Security headers → body limits → maintenance gate (section 9) → session lookup by token hash → role check → input validation → the library function, inside `tx()` when it changes data → audit entry and outbox messages in the same transaction → JSON. Errors become a plain message and a status code.

Key algorithms. *Interest:* daily simple interest on each bill's unpaid principal after its due date, rounded to rupees, posted as a ledger line before any payment, so a payment clears interest first. *Receipts:* one counter row per financial year, incremented in the same statement that inserts the payment. *Matching:* section 5. *Duplicates:* an open complaint in the same category and tower in the last 48 hours, about a shared asset (such as a lift) or with word overlap of 0.3+, is linked. *Sign-off:* the month's figures frozen as JSON with a SHA-256 fingerprint, re-checked on every view.

States	
Complaint	open → assigned → in progress → resolved → closed (rated); a reopen sends a resolved complaint back to work and is counted
Bank row	posted · queue → pending approval → resolved · duplicate · reconciled · ignored
Bill run	draft → issued (when a second person approves) · draft → rejected

API, by area. Auth: `/auth/otp`, `/auth/verify`. Resident: `/home`, `/bills`, `/pay/order`, `/pay/verify`, `/receipts/:id/pdf`, `/complaints` (+ `rate`, `reopen`). Desk: `/desk/complaints`, `/desk/bank/import`, `/desk/bank/queue`, `/desk/bank/rows/:id/post`, `/desk/approvals/:id/approve`, `/desk/bill-runs`, `/desk/report`. Payments: `/payments/razorpay/webhook` (signed). The optional-depth routes are listed in section 9.

4.3 Technology choices

Choice	Why
PostgreSQL	Transactions, row locks, unique and check constraints, triggers: the money rules live in the database
Node 20 and Express	One language with the client; small, well understood; enough for the load by a wide margin
React and Vite	One codebase for phone, desk, tablet and front page; the front page is split out so the app stays light
Render (free plan)	A live demo from the existing Dockerfile at no cost; the same image runs anywhere

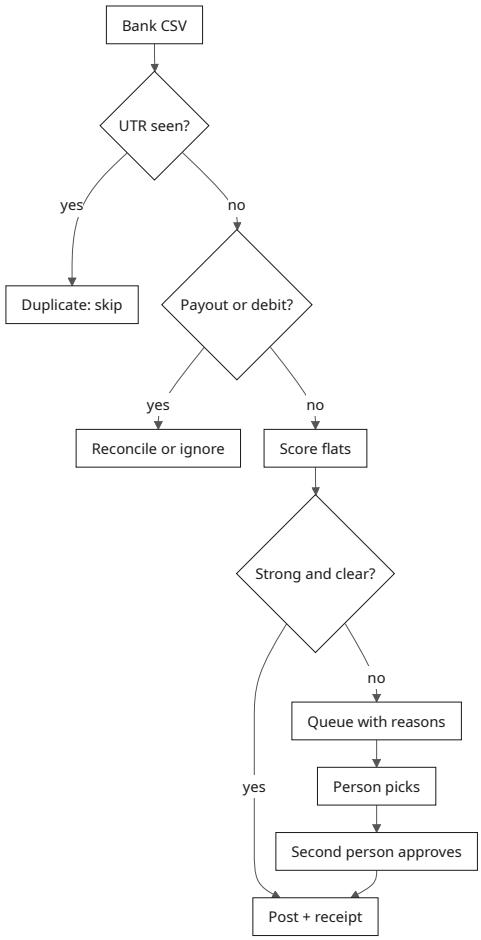
4.4 Scaling

MEASURED One process handles 270–300 requests a second against a bill-day peak of about 8, so one township has wide headroom. Normal request handling and financial locking are designed to run across several processes through PostgreSQL: sessions, rate-limit counters and the outbox live there, and all locking is in the database (section 5.4). The one exception is the demo reset's maintenance gate, which is intentionally process-local because the assignment deployment runs a single application process; a multi-process deployment would move it to shared, database-backed coordination. Next steps by growth: more processes and a connection pooler for a busier township; monthly partitions and archiving for `visitors`, `messages` and `ledger` over years; a read replica for the report and Tally export; a `township_id` with row-level security, or a database per township, to serve many. What would bind first is database write throughput for very large bill runs across many townships.

5. Financial reconciliation and safety

This is the brief's hard part, so it carries the most safeguards and the most evidence.

5.1 How a credit is decided



Evidence	Points
Flat number in the reference	+60 strong
Registered UPI ID	+50 strong
Amount equals the flat's dues	+30
Amount equals one month's bill	+20
Payer name matches a resident	+15
Part payment of what is due	+5
Far above dues (over 3×)	−40

"Strong and clear" (A5): strong evidence, a score of 80+, a lead of 30+ over the next flat, and no other flat with strong evidence. Name or amount alone never posts.

> Automated posting is restricted by matching-confidence and competing-evidence safeguards. Ambiguous or conflicting transactions are routed for human review, and a manual post takes effect only when a second person approves it. **MEASURED** In the evaluation, 0 of 140 test credits were posted to the wrong flat; that is a result on the tested statements, not a proof for every possible statement.

5.2 Payments, receipts and the ledger

The server creates the gateway order for the amount it computes (never the browser's); after checkout it checks the signature and fetches the payment from the gateway; then the payment, its ledger line, its receipt number and its alert are written in one transaction. A signed webhook covers residents who close the app mid-payment, and either path records the payment once. Receipt numbers come from a counter row updated in the payment's own transaction, not a PostgreSQL sequence, so a failed payment gives its number back and receipts have no gaps.

5.3 Matching time: an estimated −88%, and how it is calculated

> **88.2% estimated matching-time reduction on the 140-credit evaluation dataset, using the stated time assumptions (A12).** The credit counts are measured; the minutes per credit are assumptions. It is not a measured saving in a real society's staff time.

One model (`server/lib/matchtime.js`) is used by both the committee report, on real imports, and the evaluation, on statements with known flats, with the minutes in A12. On the evaluation's 140 maintenance credits (counts **MEASURED**, minutes assumed):

Step	Value
By hand: every credit at 2 minutes	140 × 2 = 280 minutes
With the app: 77 posted (0 min), 62 queued with the right top suggestion (0.5 min), 1 queued without it (2 min)	0 + 31 + 2 = 33 minutes
Saved (estimate)	(280 − 33) ÷ 280 = 88.2%, against the brief's target of 80%

The minutes are assumptions (A12); with 1 minute to confirm a suggestion instead of 0.5, the saving is about 77%. `npm run eval` prints the inputs, so other numbers can be tried.

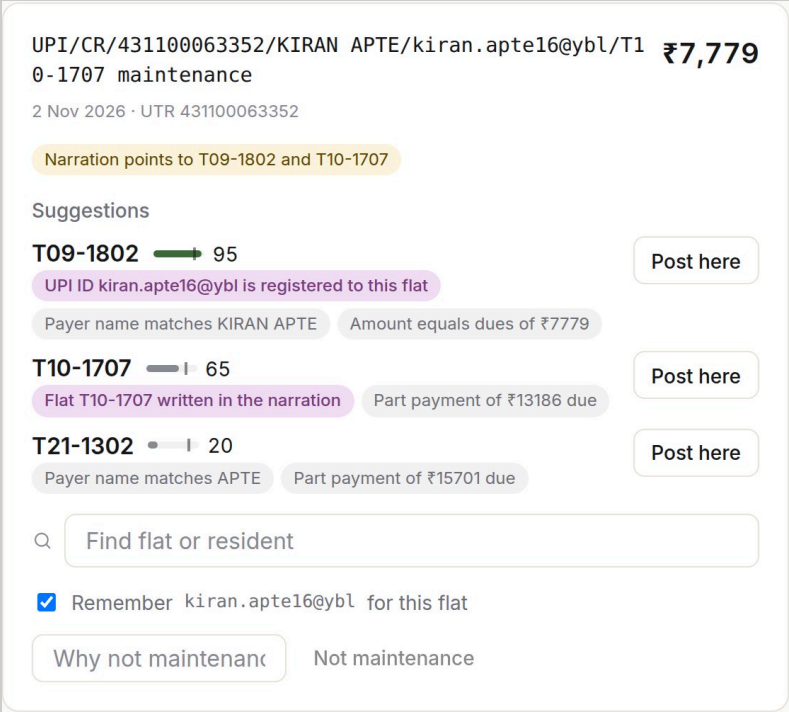
5.4 What can collide, and what stops it

Each guarantee is enforced by PostgreSQL (row locks, unique constraints, advisory locks), so it holds with many users and several processes.

Race	What stops it
The checkout callback and the webhook arrive together	Both lock the payment order row; the second returns the same receipt; `UNIQUE gateway_payment_id` backs it up
Two payments for one flat at once	The flat row is locked while interest is posted and the payment allocated
Two receipts with the same number	The counter row is incremented in the payment's own statement; its row lock serialises payments
Two statement imports, or the same one twice	An advisory lock allows one import at a time; UTRs already imported are duplicates
Two people resolve one queued credit	The bank row and approval are locked; one pending approval per row (partial unique index); no self-approval (CHECK)
A bill run retried or issued twice	`UNIQUE (flat_id, period)` with `ON CONFLICT DO NOTHING`; one open run per month; the run row locked while approved

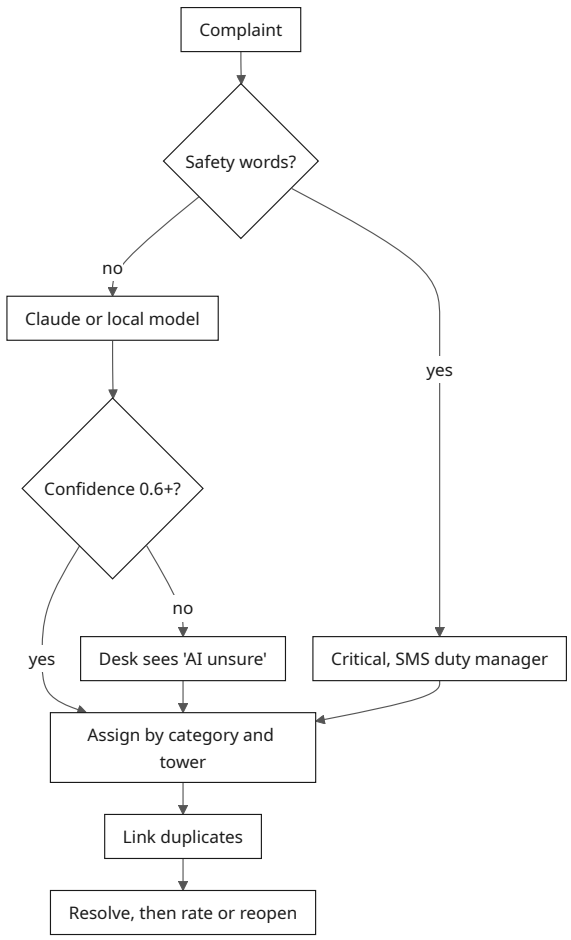
5.5 Proven by tests

- **Ambiguous credits wait for a person:** name or amount alone never posts; two strong signals for different flats queue; queued credits carry no payment and say why (`matching.test`, `reconciliation.test`).
- **Duplicate imports create nothing:** importing the same statement twice adds no payment, ledger line, credit or audit entry (`reconciliation.test`).
- **Failed transactions leave nothing behind:** a payment that fails after being written leaves no payment, ledger line or audit entry and an unchanged receipt counter; the next payment takes the next number (`reconciliation.test`).
- **Approvals are enforced:** the requester cannot approve their own post through the app or directly in the database (`api.test`, `reconciliation.test`).
- **Every money decision is audited and the audit cannot be edited:** a manual post is audited as requested, approved and recorded; adjustments are audited; the ledger, payments and audit log reject updates and deletes (`reconciliation.test`, `billing.test`, `governance.test`).



A credit whose narration names T10-1707 while its UPI ID is registered to T09-1802: two strong signals disagree, so it is queued with both flats and their evidence instead of being posted.

6. AI complaint classification and evaluation



The brief requires	Built as
Category and urgency from free text	Claude with a fixed category list and a JSON schema; refusals, invalid answers or an 8 s timeout fall back to the local model
One-tap category change	A category menu; "AI unsure" below 0.6; the AI's guess is kept
Fire, gas, lift stuck, shock always escalate	A rule before the AI, in three languages; critical urgency that cannot be lowered
Duplicates linked, never closed	Linked to the main complaint; each flat closes and rates its own

MEASURED **90 labelled complaints (EN, HI, MR, Hinglish), local model**	**Result**
Category right; right when the model says it is sure (35% of complaints)	81%; 100%
Flagged "AI unsure" for the help desk to check	64%
Hazards escalated; false alarms	14 of 14; 0

The labels follow the urgency rules given to the model, not its keyword list, so the score is not tuned to the model. **Honest confidence:** the local model discounts thin evidence (`1 − 0.5`^{n}), so one keyword gives about 50%, not 100%; it is cautious, which costs the help desk a tap, not a wrong job. **What the first run found and fixed:** Hindi and Marathi words were split on their vowel signs (50% right; now 86% and 71%); the Marathi "अडकले" (stuck) was missed; "लिफ्ट बंद" (the lift is off) and "fire lane" raised false alarms. With a Claude key, `npm run eval -- --claude` scores Claude on the same set.

7. Security, auditability and permissions

Role	Can do
Resident	Own flat only: pay, complain, approve visitors, guest codes, book, add family
Help desk	Complaints; correcting residents' details (audited)
Accounts	Bank matching, approvals, flats and ledgers, bill runs, adjustments
Committee	The report; approves bill runs; signs off months
Admin	All of the above, plus staff, tablets, settings, the audit log, the demo reset
Guard	The gate console, only on a paired tablet with a PIN

- **Sign-in and sessions:** codes and session tokens stored only as hashes; 5 minutes and 5 tries per code; at most 3 codes per number an hour (15 in demo mode); unknown numbers get the same reply; sessions last 30 days for residents and 12 hours for staff.
- **Inputs and outputs:** every route validates its body; CSV exports neutralise spreadsheet formulas; uploads must be real images; a strict Content Security Policy.
- **Audit:** every money decision, correction and reset is written to an append-only audit log (payments and approvals in the same transaction as the change itself); the database role the app uses cannot change it.
- **Secrets and data:** only `.env.example` is committed; a scan of all commits found no keys, tokens or `.env` files; all names, numbers and UPI IDs in the sample data are invented. DPDP: consent, data download, correction, erasure (accounts kept as the law requires), visitor photos deleted after 90 days.

8. Testing and reproducibility

MEASURED From a fresh clone (`git clone`, `npm ci`): **63 server tests** and **6 browser tests** pass, and `npm run eval` gives identical results on repeated runs and matches the committed `docs/evaluation.md`. The evaluation data is in the repository (`server/eval/`), and the statements are generated from fixed seeds.

Command	What it proves
`npm test`	63 server tests: billing, interest, payments, matching, reconciliation safety, complaints, roles, governance, privacy, the demo reset, and the evaluation floors
`npm run test:e2e`	6 browser tests in headless Chrome: the main flows, and no browser errors on any page

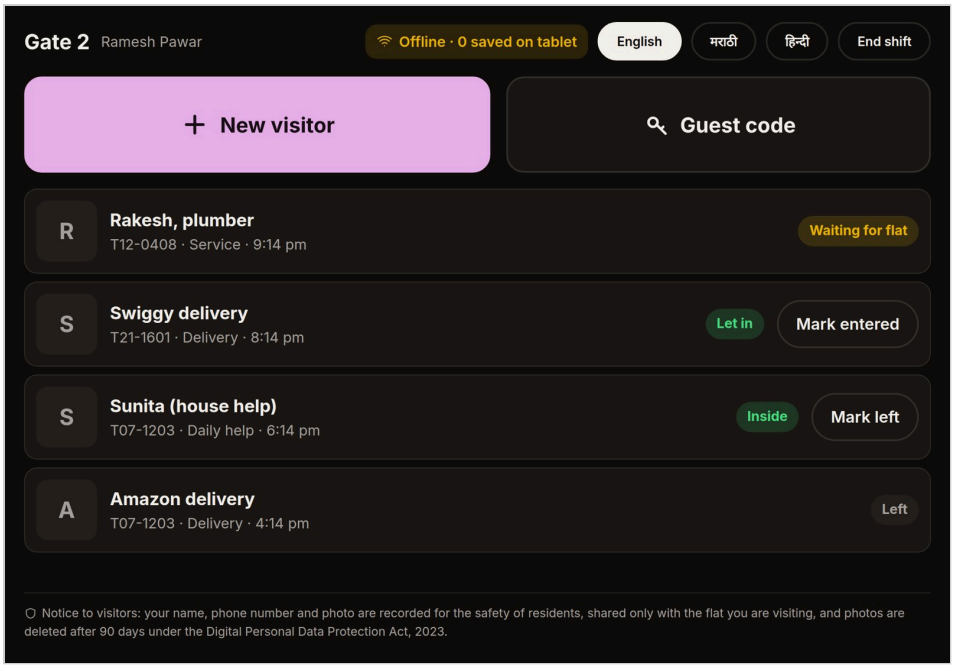
`npm run eval`	Sections 5.3 and 6, written to `docs/evaluation.md`
`npm run load:seed` then `npm run load:run`	The load test below

Load test configuration. A seeded township at full size: 2,400 flats in 28 towers, about 4,200 residents, a year of bills and payments, 6,000 complaints, 393,000 gate entries, 150,000 messages. Traffic: 85% residents, 10% gate, 5% desk, at fixed arrival rates up to 40 requests a second (five times the bill-day peak of about 8), then users clicking without pause to find the ceiling; then the heavy jobs are timed. **MEASURED** on one laptop (Intel Core i3-1220P, 12 threads, 16 GB RAM) with one app process and PostgreSQL 16 on the same machine: 95% of requests under 28 ms at 40 a second with no errors; a ceiling of 270–300 requests a second; a bill run for every flat in 4.1 s; 2,400 bank credits in 17.5 s; the committee report in 1.7 s. These numbers depend on the hardware; the targets (95% under 300 ms, a bill run under 30 s) are checked by the script, which fails if one is missed.

9. Optional depth and additional features

Built from the brief's "must have", tested, and secondary to the core:

- **Gate visitor management:** a paired tablet with a guard PIN, visitor photo, a push to the flat, approve or deny, guest codes; works through network drops (entries queue on the tablet with a client id and sync once; `ON CONFLICT (client_id) DO NOTHING` keeps one); very large buttons on a dark screen; the DPDP notice on screen; photos deleted after 90 days.
- **Amenity booking:** slots and capacity, limits, fees on the bill, free cancellation until a cut-off, closed days that refund. Two flats cannot take the last place at once: booking locks the amenity row and re-counts inside the lock.
- **Resident extras:** family and tenant members, notices, push alerts, English, Marathi and Hindi, and an installable web app with a Download app button (the browser's prompt, or Safari's steps; hidden once installed).
- **Office extras:** register import from Excel with a preview, Tally export, the message outbox, data requests, the audit log.
- **Demo reset** (demo mode, admins only): restores the sample township; other requests get "resetting demo data, try again" and requests already running finish first; workers pause; everyone signs in again; every reset stays in the audit log. The pause is intentionally process-local because the deployment runs a single application process (section 4.4); with several processes it would move to the database. **If a reset fails:** the whole rebuild, schema included, is one PostgreSQL transaction, so it is rolled back and the previous demo state is kept exactly, earlier reset records included; the pause is always lifted and the background workers carry on by themselves; the failure is recorded in the audit log (`demo.reset_failed`) and the server log, and the admin is told nothing was changed. `demo-reset.test` forces a failure after the data is rebuilt and checks all of this.
- **Front page** (`/welcome`): explains the product in seven chapters; loaded separately, so the app carries none of it.
- **Optional tooling, not a completed production deployment:** Terraform for AWS Mumbai (written and checked in CI, never applied) and the load-test scripts.



Gate tablet with the network down: large buttons, dark for night, the offline state, the DPDP notice.

10. Limitations and future improvements

- **Cut:** WhatsApp (needs Meta business approval); real SMS and payments (mocked, as the brief allows, following the real providers' steps, so switching is configuration); native apps; technician sign-in.
- **Limits:** the live demo is on Render's free plan (it can sleep, though a scheduled ping keeps it awake; uploaded photos are lost on restart; the free database expires after 30 days, around 4 December 2026). Reviewers share one demo township; an admin can reset it. Without a Claude key the classifier is keyword-based: 81% right, cautious, and it flags what it is unsure of. Load-tested on one process and one laptop. The interest rate is a demonstration rate (A2).
- **Next:** measure Claude on the labelled set and grow the set from the help desk's corrections; remember UPI IDs by default, the largest lever on automatic matching; WhatsApp alerts through the same outbox; bank feeds instead of CSV uploads; for several processes, keep the maintenance flag in the database.

Appendix A. The assignment, point by point

The assignment asks for	Where it is met
The core build working end to end	Sections 2 and 3; the brief's three flats reproduce its table, and a test checks them
Main flows on realistic sample data; no dead buttons	A sample township loads on first start; 6 browser tests click through the main flows and fail on any page error
Frontend, backend, database, sign-in, roles	Sections 4 and 7
The AI part works, including when wrong or unsure	Section 6
Polished UI; works on a phone; loading, empty and error states	Section 2.1
Tests; no secrets; validated inputs; access control	Sections 3, 7 and 8

TDD: GitHub URL and live link; architecture and why; data model, flows, AI; UI decisions; cuts and next	Header; sections 4, 5, 6, 2.1 and 10
deployment.md: run locally, how it is deployed, variable names, demo logins	deployment.md
Only the brief; every assumption labelled	Section 1.3
Commit history kept; no secrets or real personal data	Each change is its own described commit; section 7

Appendix B. Questions a reviewer might ask

Why not use an LLM for bank matching too?

A wrong match is a wrong flat's money and a committee's trust. Rules give the same answer every time and explain every point, so they can be audited. An LLM could suggest flats in the queue later, never post.

What happens when Claude is down, slow or wrong?

Down or slow (8 s): the local model answers and the complaint is saved. Wrong: the help desk sees the confidence and changes the category in one tap; the AI's guess is kept to measure accuracy. Dangerous: the safety rule has already escalated it.

Can a payment end up on the wrong flat?

Automatic posting needs strong, unambiguous evidence and no rival; everything else goes to a person, and a manual post needs a second person. The evaluation measured no wrong posts in 140 test credits; that is a result on the tested data, which is why the queue and the approvals exist.

How exactly is the 88% calculated?

Section 5.3. It is an estimate on the 140-credit evaluation dataset, not a measured staff saving: 280 minutes by hand against 33 with the app, using the minutes in A12, from `npm run eval`; with 1 minute per confirmation it is about 77%. The committee report applies the same model to real imports.

What if the same statement is uploaded twice, or a payment fails half-way?

Rows whose UTR is already imported post nothing; a failed payment rolls back with its ledger line, audit entry and receipt number. Both are tested (section 5.5).

Can two flats book the last slot?

No: booking locks the amenity row and re-counts inside the lock. The same pattern protects payments, receipts, bill runs and approvals (section 5.4).

Why 21% interest?

The brief gives no rate. 21% is a configurable demonstration rate, to be confirmed against the federation's adopted bye-laws before production (A2).

How can the committee trust the report?

Every rupee is a ledger line the database will not let anyone edit; money decisions need two people; receipts have no gaps; a signed month is frozen with a fingerprint checked on every view.

Why so much beyond the core build?

The core build came first and has the most tests and the evaluation. The brief lists the rest as optional depth and the assignment counts well-judged depth in favour, so it was built on the same foundations, tested, and kept secondary (section 9).

What if two reviewers use the live demo?

They share one township. An admin can reset it from the Admin page; other users are paused for the moment it takes, and every reset is kept in the audit log.

Where did AI tools help, and where were the judgements yours?

AI drafted code, tests and copy faster and found gaps. The rules (matching thresholds, two-person checks, the safety rule above the model), the assumptions, the evaluation design and the decision to measure rather than claim were my judgements.